Godot + GDScript 大规模行人、GPU 计算与未来扩展研究

Executive Summary

结论先说在前面：`Godot + GDScript` 不是“天花板只到这里”，但如果继续沿着“SceneTree 里很多活体 NPC 节点 + 主线程逐个更新”的路径走，性能上限确实会很早撞墙。Godot 4.x 已经提供了 `RenderingDevice` 与 compute shader，社区里也确实有人在 Godot 里用 GPU 做 boids、slime、wave 之类的大规模并行模拟；但这些成功案例几乎都集中在“数据并行、规则统一、主要服务于视觉表示”的问题上，而不是把完整 NPC AI、导航、状态机、伤亡反应全部搬进 GPU [1][2][9][10]。

对开放世界行人系统，更现实的路线不是“把全部 NPC 脑子搬到 GPU”，而是走混合架构：CPU 端用数据导向结构维护 authoritative gameplay state，近景少量高保真代理继续走完整逻辑；中远景用分页、分层、instancing、shader/compute 驱动的轻量表示；最热的纯数据阶段再下沉到 `servers`、`WorkerThreadPool`、`GDExtension/C++` 或 compute shader。行业主流也是这个方向，而不是“所有 NPC 行为都在 GPU 上算” [4][5][14][15]。

Key Findings

- **Godot 4.x 的 GPU 计算能力是真实可用的**：官方文档明确支持 compute shader，但要求 `Forward+` 或 `Mobile` renderer，兼容渲染器不支持，而且本地 `RenderingDevice` 在 headless 模式下不可用 [1][2]。
- **GPU 更适合规则统一的大规模数据并行，不适合完整 NPC 大脑**：Godot 官方同时强调 SceneTree 不是线程安全热区，`servers` 更适合成千上万实例，而 `MultiMesh` 适合批量渲染，但需要空间分块，因为实例级剔除有限 [3][4][5][6]。
- **社区里已经有人在 Godot 里做 GPU 群体模拟**：例如 compute shader demo、Compute Shader Plus 的 boids/slime 示例，说明“用 GPU 算群体”这件事在 Godot 里不是空想 [9][10]。
- **但 GPU 不是免费午餐**：一旦你需要频繁把 GPU 结果回读给 CPU，或者让 GPU 逻辑强依赖导航、物理查询、命中判定、复杂状态机，收益会被同步和数据搬运抵消。已有 Godot 项目明确记录过 local `RenderingDevice` 的异步路径因为 CPU/GPU 传输成本反而不值 [2][11]。
- **业界主流是混合分层架构**：Unreal Mass 与 Unity 的 crowd/culling 路线本质上都在强调数据导向、表示层 LOD、近远景行为分级，而不是让每个远处 NPC 都继续作为完整 Actor/GameObject 存在 [14][15]。

Detailed Analysis

1. Godot 官方到底支持到什么程度

Godot 4.x 官方文档明确提供 compute shader 教程，入口就是 `RenderingDevice`。但这条能力有几个硬边界：

1. 只在 `Forward+` 和 `Mobile` renderer 下可用，`Compatibility` renderer 不支持 compute shader [1]。
2. 官方 `RenderingDevice` 文档明确写了：本地 `RenderingDevice` 可以在“单独线程”创建和使用，但它不能跨线程共享资源，而且 **在 headless 模式下不可用** [2]。
3. Godot 官方线程文档同时强调：把操作挪到线程上不代表一定会更快，涉及渲染服务器、GPU、独占资源时要谨慎，线程同步也会有代价 [3]。

这三条合起来意味着：你当然可以在 Godot 里做 GPU/compute 原型，但它不是“把现有 GDScript NPC 更新函数直接丢给显卡”这么简单。你需要重新定义数据布局、同步边界、authoritative state 的归属，以及 profiling 方法本身。

2. Godot 官方推荐的大规模对象路线，其实已经在暗示答案

Godot 官方文档对大规模对象的建议非常一致：

- `Using servers` 明确说，如果场景系统成了瓶颈，直接用 servers 更合适，尤其是需要持续处理的成千上万实例时 [4]。
- `Optimization using MultiMeshes` 强调 `MultiMesh` 能把大量对象压成很少 draw calls，但代价是整批实例的可见性和批处理粒度要自己设计 [5]。
- `MultiMesh` 类文档进一步强调，它把所有实例当成一个整体处理，想要有效剔除就必须自己按区域拆成多个 `MultiMesh` [6]。
- 同一组官方资料里还直接提到：如果要进一步追求效率，可以考虑 `GDExtension` 和 C++，配合 `RenderingServer.multimesh_set_buffer()` 一类底层接口 [5][7]。

这套建议本质上是在说：Godot 能跑很多东西，但前提是你别把“很多东西”继续建模成很多活跃 `Node3D`。真正能扩的不是节点数，而是“数据页 + 批量表示 + 近远景分层”。

3. 有没有人真的在 Godot 里用 GPU 做群体/并行模拟

有，而且不止一个方向。

3.1 官方与官方生态已经给出正面证据

- Godot 官方教程《Animating thousands of fish with MultiMeshInstance》本身就是一个非常典型的“不是每条鱼一个节点，而是批量实例 + shader 驱动”的案例，目标就是让大量单位在低端硬件上也能跑得动 [8]。
- 官方 demo 项目 release 日志里已经出现 compute shader 示例，例如 “Compute Texture Demo” 和基于 compute shader 的高度图案例，说明 compute 并不是实验室功能，而是官方在持续提供示例 [9]。

3.2 社区里已经有人把它做成插件/范式

`Compute Shader Plus` 这个开源项目专门给 Godot 4 做 GDScript 侧更易用的 compute shader 封装，仓库自带的 demo 就包含 `boids simulation`、`slime mold`、`parallel reduction` 等案例 [10]。这说明两件事：

1. “Godot 里用 GPU 做群体运动/并行更新”是可行的。
2. 这类案例的共同点依然是 **统一规则、纯数据、弱分支、少回读**。

3.3 但也有人明确踩过坑

`2Retr0/GodotOceanWaves` 这个 Godot 4 海浪项目在 README 里直接写了：虽然 local `RenderingDevice` 理论上支持异步，但在他们的场景里启用后因为 CPU/GPU 数据传输开销，反而不值得 [11]。这正是 crowd GPU 化最需要警惕的点：只要结果每帧都要拿回 CPU，再喂给导航、战斗、物理、事件系统，收益可能很快蒸发。

4. 哪些行人工作适合 GPU，哪些不适合

4.1 适合 GPU 的部分

如果你的目标是“城市里看起来人很多，而且大多数时候只是合理地流动、散开、避让、抖动、呼吸、切换朝向”，那么下面这些很适合 GPU 或至少适合更底层的数据并行层：

- 中远景 crowd 的位置积分与速度积分
- 规则比较统一的 flee / wander / flocking / flow-field 跟随
- 近似的局部避让或方向扰动
- 动画相位推进、朝向插值、实例 transform 生成
- impostor / billboard / simplified actor 的姿态选择
- crowd occupancy / density field / danger field 的扩散与采样

这些任务的共同点是：状态结构规整、可以批量算、单体错误代价不高、视觉上“像”就行。

4.2 不适合 GPU 的部分

如果你说的是“真正的 NPC 行为”，那核心难点通常还在 CPU：

- 武器命中、爆炸、伤亡、事件链传播的 authoritative 判定
- 与导航系统、可达性、路口语义、地形约束的深度耦合
- 复杂分支状态机、任务切换、目标选择
- 物理射线、碰撞、动态障碍处理
- 需要稳定可回放、可调试、可复现的 gameplay state

Godot 官方导航文档也在提醒同一个方向：`NavigationAgent` 每次路径更新、RVO avoidance 都有成本，参数不当会明显增加负担；`NavigationObstacle` 也不是越多越好，因为 avoidance 需要额外计算 [12][13]。这类逻辑和 GPU 的 SIMD 风格天生不太合。

5. 业界大量 NPC / crowd 的常见架构是什么

公开的一手资料显示，主流路线基本是“数据导向 CPU + 表示分层 + GPU 做显示或局部并行”。

- Unreal 的 `Mass` 文档把它定义成 data-oriented / ECS 风格系统，核心目标就是在大规模实体下保持可扩展；其 representation/LOD 还会把实体分成 high-res actor、low-res actor、instanced static mesh、hidden 等不同表示层级 [14]。
- Unity 的 `CullingGroup` 官方示例甚至直接拿 crowd 举例：屏幕外角色用简化模拟；可见近处角色才切回完整 GameObject、全质量 AI 和动画 [15]。

这说明一个非常关键的现实：**业界不是靠“每个 NPC 都继续做完整逻辑”来堆规模，而是靠“先承认不同距离的人不该花同样的钱”来堆规模。**

6. 回到你的项目：`Godot + GDScript` 现在是不是已经到头了

不是到 Godot 的头了，但很可能已经接近“以当前建模方式继续堆功能”的头了。

更准确地说：

- **不是 Godot 不行**：Godot 4.x 已有 compute、servers、MultiMesh、线程、`GDExtension` 这些工具 [1][2][4][5][7]。
- **不是显卡白放着**：GPU 完全可以接一部分 crowd 负载，尤其是中远景统一行为和视觉表示 [1][8][9][10]。
- **但纯 GDScript + SceneTree-heavy 的 NPC 架构会很快触顶**：因为热点最终会卡在主线程对象管理、同步、导航、批量提交和大量 branchy gameplay logic 上 [3][4][5]。

所以更现实的判断不是“Godot+GDScript 只能做到这样”，而是：

> `Godot + GDScript` 很适合做编排层、测试层、工具层、规则原型层；
> 但当你进入“开放世界高密度行人 + 未来车辆 + 真实建筑 + 复杂事件”的组合区间时，最热的那层通常必须走数据导向重构，必要时下沉到 servers、C++/GDExtension，或用 compute shader 接管其中一部分统一规则的仿真。

7. 对你这类城市项目，最务实的 GPU 使用方式

如果目标是“人口密度不退化，还要给车辆和真实建筑留预算”，最务实的顺序不是直接 GPU 化全部 NPC，而是：

1. **先保持 authoritative gameplay state 在 CPU**
   伤亡、恐慌、命中、事件传播、可回放状态，先别交给 GPU。
2. **把 far/mid crowd 彻底从“人”改成“数据页”**
   不是很多 `Node3D`，而是 page-based state + instancing。
3. **优先把统一规则的中远景运动交给 GPU 或更底层数据层**
   例如 far-tier 漫游、flee budget 消耗、方向场跟随、简单局部避让。
4. **避免每帧 GPU -> CPU 回读**
   最好让 GPU 结果直接服务于可视化表示，CPU 只偶尔抽样或在层级切换时同步。
5. **把近景高保真代理数量钉死在很小范围**
   真模型、真动画、复杂状态机、复杂碰撞，只留给核心圈。
6. **如果要继续深挖性能，优先考虑 hot path 下沉到 `GDExtension/C++`**
   这是 Godot 官方资料里也反复暗示的扩展方向 [5][7]。

Areas of Consensus

- Godot 4.x 具备做 GPU 计算和大规模实例化的技术基础 [1][2][5][6]。
- 真正要扩大量级，必须放弃“每个远处 NPC 都是完整节点 + 完整 AI”的思路，转向分层表示与数据导向 [4][5][14][15]。
- GPU 最适合统一、规整、可批量的 crowd 任务，不适合直接承担完整 gameplay authority [1][2][3][11]。

Areas of Debate

- **Godot 原生 GDScript + compute 能走多远**：能做出不错的中远景 crowd 原型，但如果要承载更复杂的车辆/建筑/动态障碍联动，是否最终仍要落到 `GDExtension/C++`，不同团队的答案会不一样 [7][10]。

- **GPU crowd 是否值得为当前项目引入**：如果当前最重的仍是 first-visit streaming / mount / CPU side world prep，那么先做 GPU crowd 不一定是 ROI 最高的一刀。这需要结合项目自身 fresh profiling 判断，而不是凭感觉 [2][3][11]。
- **Godot 里 full-GPU crowd 的成熟度**：有示例、有插件、有原型，但距离 Unreal Mass / Unity DOTS 那种“成体系的官方游戏层解决方案”仍有差距 [10][14][15]。

Practical Recommendation

如果把你的担忧翻译成工程语言，我会给出这条路线：

1. 继续把当前 `M10` 的主瓶颈压在 CPU 冷路径和 streaming 主线程上，因为这是现阶段最直接的红线来源。
2. 并行开一个很小的 GPU 原型，不碰 authoritative gameplay，只验证一件事：
   - `far-tier crowd flee/wander integration + instanced visual commit`
3. 这个原型如果成立，再决定后续是：
   - 继续纯 Godot `RenderingDevice` + GDScript 包装，
   - 还是转 `GDExtension/C++` 做真正的高热层。

换句话说，**你现在不需要悲观到“以后加车辆和建筑肯定跑不动”**；但也不能乐观到“再抠一点 GDScript 细节就能无限扩”。真正的分水岭是架构升级，不是微调。

Sources

[1] Godot Engine Documentation, “Using compute shaders.” 官方文档，说明 Godot 4.x compute shader 的使用条件与 renderer 限制。
https://docs.godotengine.org/en/stable/tutorials/shaders/compute_shaders.html

[2] Godot Engine Documentation, “RenderingDevice.” 官方类文档，说明 local `RenderingDevice` 可在单独线程使用，但资源不跨线程共享，且在 headless 模式下不可用。
https://docs.godotengine.org/en/stable/classes/class_renderingdevice.html

[3] Godot Engine Documentation, “Thread-safe APIs.” 官方文档，强调线程同步成本与 SceneTree / GPU 相关注意事项。
https://docs.godotengine.org/en/stable/tutorials/performance/thread_safe_apis.html

[4] Godot Engine Documentation, “Using Servers.” 官方文档，建议在场景系统成为瓶颈时考虑更底层的 servers，尤其是需要持续处理的大量实例。
https://docs.godotengine.org/en/stable/tutorials/performance/using_servers.html

[5] Godot Engine Documentation, “Optimization using MultiMeshes.” 官方文档，说明 `MultiMesh` 的 draw-call 优势与大规模对象的推荐用法，并提到 `GDExtension/C++` 与底层 buffer API 的进一步优化空间。https://docs.godotengine.org/en/stable/tutorials/performance/using_multimesh.html

[6] Godot Engine Documentation, “MultiMesh.” 官方类文档，说明 `MultiMesh` 会将实例作为整体处理，因此需要按空间拆分来实现更好的剔除。
https://docs.godotengine.org/en/stable/classes/class_multimesh.html

[7] Godot Engine Documentation, “What code model should I use? / GDExtension.” 官方 FAQ，指出 `GDExtension` 适用于需要原生性能或现有 C/C++ 库的场景。
https://docs.godotengine.org/en/stable/about/faq.html#what-code-model-should-i-use

[8] Godot Engine Documentation, “Animating thousands of fish with MultiMeshInstance.” 官方教程，展示大量个体通过批量实例与 shader 驱动表现的路线。
https://docs.godotengine.org/en/stable/tutorials/performance/vertex_animation/animating_thousands_of_fish.html

[9] godotengine/godot-demo-projects, Release notes for official demo projects. 官方 demo 发布日志，包含 compute shader 相关 demo（如 Compute Texture Demo、compute shader 高度图）。
https://github.com/godotengine/godot-demo-projects/releases

[10] Nice Effort, “Compute Shader Plus.” 开源项目源码与 README，提供 Godot 4 的 compute shader 封装，并包含 boids/slime 等 demo。https://github.com/niceeffort/compute-shader-plus

[11] 2Retr0, “GodotOceanWaves.” 开源项目 README，记录 local `RenderingDevice` 异步路径因 CPU/GPU 传输成本而不划算的工程经验。https://github.com/2Retr0/GodotOceanWaves

[12] Godot Engine Documentation, “Using NavigationAgents.” 官方文档，说明导航代理参数、路径更新和 avoidance 的使用边界与成本注意事项。
https://docs.godotengine.org/en/stable/tutorials/navigation/navigation_using_navigationagents.html

[13] Godot Engine Documentation, “Using NavigationObstacles.” 官方文档，说明 avoidance 相关对象的用途与额外开销边界。
https://docs.godotengine.org/en/stable/tutorials/navigation/navigation_using_navigationobstacles.html

[14] Epic Games Documentation, “Mass Entity Overview” / “Mass Gameplay Overview.” 官方文档与检索摘要，说明 Unreal Mass 的 data-oriented / ECS 路线，以及 representation LOD 会在 high-res actor、low-res actor、ISM、hidden 等层级间切换。https://dev.epicgames.com/documentation/en-us/unreal-engine/overview-of-mass-entity-in-unreal-engine
https://dev.epicgames.com/documentation/en-us/unreal-engine/overview-of-mass-gameplay-in-unreal-engine

[15] Unity Documentation, “CullingGroup API.” 官方文档，示例明确以 crowd 为例，强调屏幕外角色可用简化模拟，近景才切回完整 GameObject、AI 和动画。
https://docs.unity3d.com/ScriptReference/CullingGroup.html

Gaps and Further Research

- 还没有针对你这个项目的真实渲染器模式做 fresh GPU profiling。当前很多自动化测试跑的是 headless + dummy renderer，它更能证明 CPU 主线程是否先爆，但不能直接量化真窗口下 GPU 的饱和度。
- 还没有为当前项目做“GPU crowd 原型”的 A/B 实验，因此不能直接断言引入 compute 之后一定会有多少收益。
- 如果下一轮真的要验证，最值得先做的不是 full NPC GPU 化，而是一个最小原型：
  - `far-tier crowd page`
  - `uniform flee/wander update`
  - `no per-frame readback`
  - `MultiMesh / shader-driven visual`